The Journal of Open Source Software

lucie: An Improved Python Package for Loading Datasets from the UCI Machine Learning Repository

Kenneth Ge [1¶], **Phuc Nguyen** [1], and **Ramy Arnaout** [1]

1 Department of Pathology, Beth Israel Deaconess Medical Center in Boston, MA, USA. ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 17 January 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Abstract

The University of California–Irvine (UCI) Machine Learning (ML) Repository (UCIMLR) is consistently cited as one of the most popular dataset repositories, hosting hundreds of high-impact datasets. However, a significant portion, including 28.4% of the top 250, cannot be imported via the *ucimlrepo* package that is provided and recommended by the UCIMLR website. Instead, they are hosted as .zip files, containing nonstandard formats that are difficult to import without additional ad hoc processing. To address this issue, here we present *lucie*—load University California Irvine examples—a utility that automatically determines the data format and imports many of these previously non-importable datasets, while preserving as much of a tabular data structure as possible. *lucie* was designed using the top 100 most popular datasets and benchmarked on the next 130, where it resulted in a success rate of 95.4% vs. 73.1% for *ucimlrepo*. *lucie* is available as a Python package on PyPI with 98% code coverage.

Introduction

Dataset accessibility is critical to advancement of machine learning research, across disciplines (Chinn et al., 2023; Dey et al., 2023; Koch et al., 2021; Mayfield et al., 2017; Miao et al., 2024; Yin & Berger, 2017). The UCI ML Repository(Markelle Kelly, 2024) (UCIMLR) is considered one of the most popular machine-learning dataset repositories(Badr, 2019). However, many of the datasets it contains are old, containing custom formats that cannot be imported via the built-in *ucimlrepo* Python package or API(*Uci-Ml-Repo/Ucimlrepo*, 2024) or via the "Import to Python'' button on the UCIMLR (Fig.Figure 1). Although undoubtedly useful, UCI datasets are known to often contain missing fields, missing headers, not-a-number (NaN) columns, and most importantly, nonstandard data formats. Among the top 100 most popular datasets, 16 are not importable using *ucimlrepo*. Among less popular datasets, the proportion that are not importable is even higher, possibly because datasets that are easier to use get more downloads. For datasets ranked 101-250, over a third cannot not be imported via *ucimlrepo* (see Results).

A small fraction of these "dirty''(*Reuters-Text-Categorization/Reuters_loader.py at Master · Jchavando/Reuters-Text-Categorization · GitHub*, n.d.) datasets have been cleaned and mirrored on other dataset repositories such as Kaggle, Huggingface, or various GitHub repositories. Examples include Diabetes (UCMLR ID #34) on Kaggle(*UCI Diabetes Data Set*, n.d.), the Reuters-21578 Text Categorization Collection (#347) on HuggingFace(*Ucirvine/Reuters21578 · Datasets at Hugging Face*, 2023), and Movie (#132) on GitHub(Černoch, 2023). However, mirroring still leaves three major problems:

- The vast majority of the non-importable datasets still lack cleaned mirror versions,
- Importing these datasets via mirrors still requires manual identification and an ad-hoc per-dataset approach, and

- The work required to track down datasets via mirrors or other sites diminishes the "all-in-one-place" value of the UCIMLR.

In our own work, we noticed that many of the problematic datasets, although differing from each other in data format and layout, share many common patterns in how they were constructed. We exploited these patterns to create *lucie*–load UCI examples– a smart-import tool that follows a nearly identical interface as *ucimlrepo* and automatically guesses the data format to import previously non-importable datasets. Then, if possible, it tries to coerce the data into a dataframe using the popular Python *pandas* package, just as *ucimlrepo* does.

Methods

Code and Datasets

ucimlrepo version 0.0.7 was installed using pip (`pip install ucimlrepo`). We determined dataset popularity rankings based on the results given by the "sort by num views, desc" criteria on the UCIMLR (accessed August 26, 2024). After saving these results, we attempted to import each of the top 100 using *ucimlrepo*. This identified 16 that were non-importable: UCMLR IDs #5, 34, 51, 102, 113, 121, 125, 132, 137, 228, 236, 240, 280, 321, 331, and 502 (alphabetically).

We downloaded these datasets using the download-as-zip function on the UCIMLR, and manually identified the following patterns in their construction:

- 5 datasets contained tabular data, in the form of `.data'' files`,.xlsx'' files, or some other clearly tabular data format;
- 4 datasets contained many ".txt'' files, some of which contained metadata and some of which contained tabular data;
- 4 datasets contained highly irregular data formats, such as HTML or XML with hyperlinks representing a relational database;
- 2 datasets contained nested archives, where each folder represented a column and each file was an entry; and
- 1 dataset contained only extension-free files, some of which contained metadata and some of which contained tabular data stored in a delimiter-separated format.

We concluded that if these patterns occurred at least once, there was a good chance they would occur again, i.e. in datasets beyond the top 100.

DRAFT

The Journal of Open Source Software

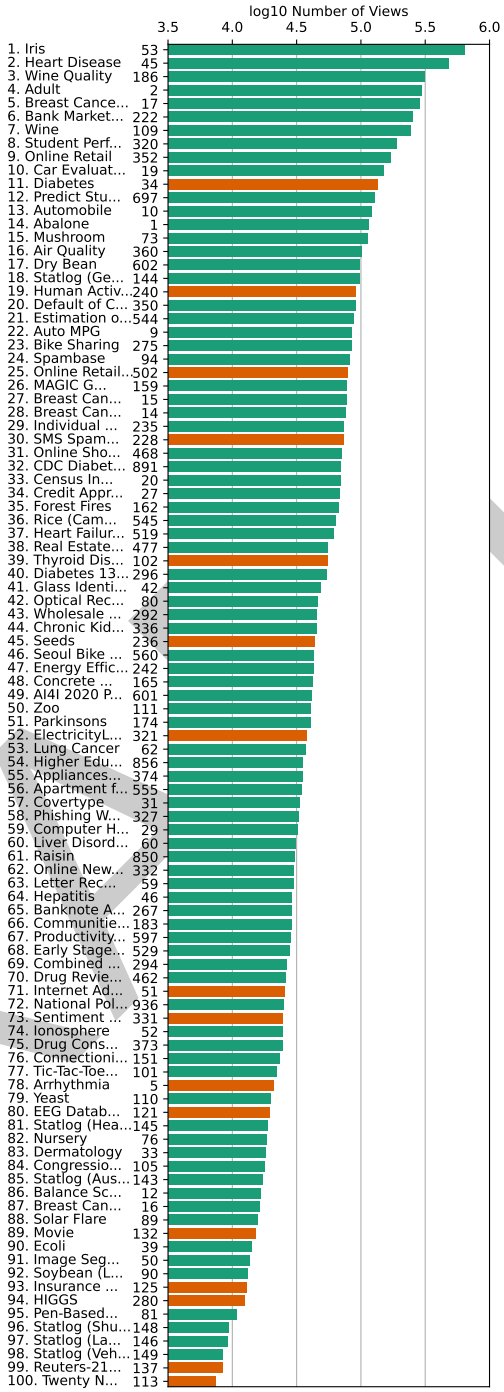

Figure 1: The 100 most popular UCIMLR datasets, ordered by number of views and presented using a log10 scale. Green=datasets that *ucimlrepo* successfully imports; red=*ucimlrepo* failures. The UCIMLR ID# is displayed along with the name and rank. Note the correlation between success and popularity (more red bars toward the bottom).

We constructed our import algorithm to be as general as possible, while minimizing any false positives (which we define to be interpreting extraneous files as data). *lucie*'s import function returns a dictionary, with keys representing the names of any files identified as tabular data (including archives), and values being corresponding *pandas* dataframes. If that is not possible, as a fallback, it instead returns a dictionary representing the directory structure and its

contained files for any identified data archives; however, in practice, no such failure mode was detected among the top 100 datasets. Finally, we also modified the `fetch_ucirepo` function to return a tuple, with the first element being `"uci"` or `"custom"`, representing whether the dataset was imported using the built-in function vs. our custom import algorithm, respectively.

Algorithm (Fig.Figure 2)

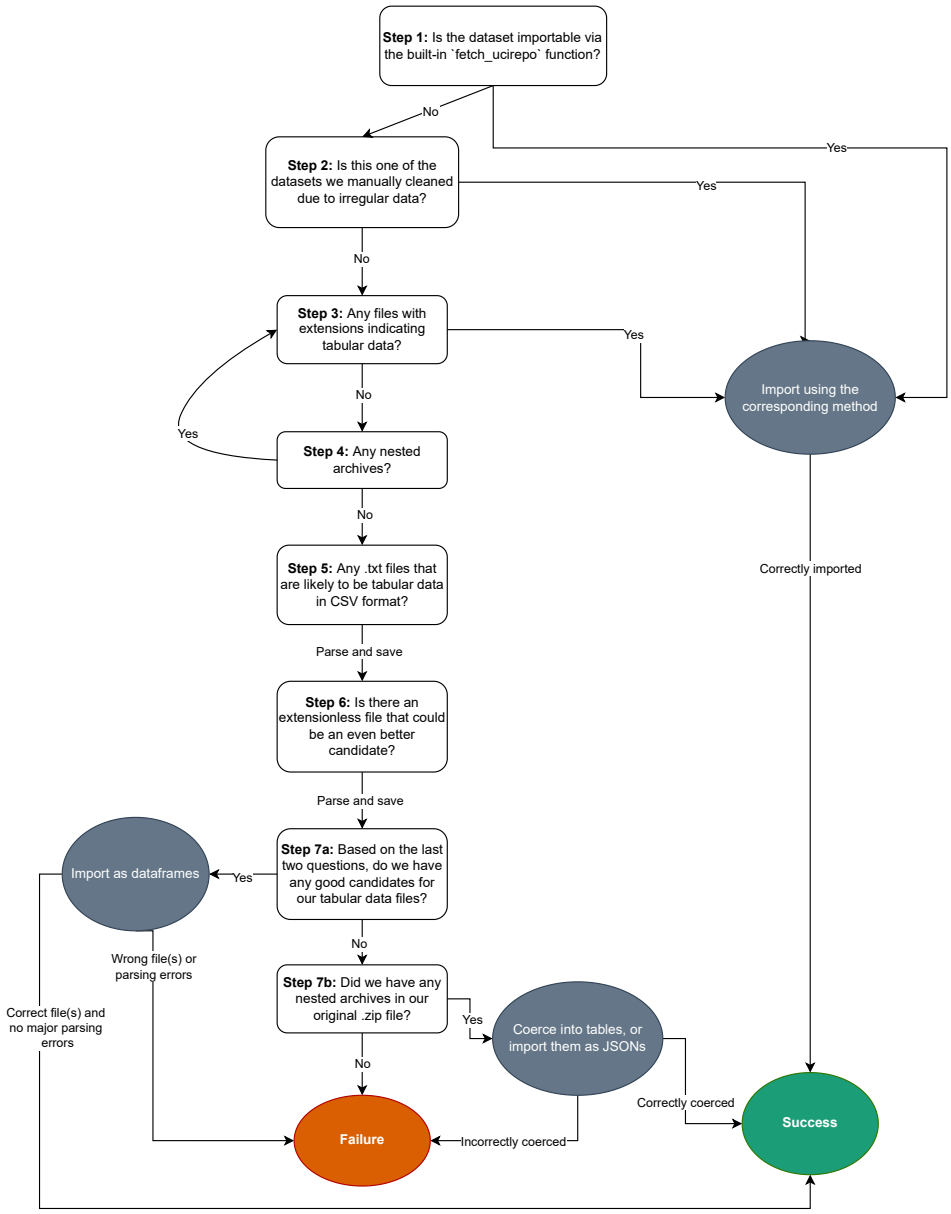

Figure 2: Flowchart of the lucie algorithm, with question boxes corresponding to the steps described in the text. The green oval indicates success, the red oval indicates failure, and the gray ovals indicate an action.

Step 1 The import algorithm first queries the UCIML API to determine if a given dataset can already be imported using the built-in `fetch_ucirepo` function. If so, it returns a tuple containing the string `"uci"` and the result. If not, we use BeautifulSoup(Richardson, 2007) to scrape the UCIMLR for the download link for the corresponding ".zip" file. Then, we extract this archive.

Step 2 For four repositories containing highly irregular data formats (EEG with ID #121, Diabetes #34, Movie #132, and Reuters-21578 #137), we found pre-cleaned mirrors and execute custom import code; *lucie* uses these, treating them as special cases.

Step 3 For the rest of the datasets, we start by looking for any files with extensions that clearly identify them as tabular data, such as ".data", ".xlsx", ".tsv", ".csv", etc. If found, we read them all and return them in the dictionary format described above. Since some of these files may have an irregular shape (i.e. some rows have more columns than others), we use a custom `read_csv` function that removes the requirement that the data be rectangular.

Step 4 If no such files are found, we then look for any nested archives. Some datasets contain multiple nested archives; for example, the "Twenty Newsgroups" dataset contains a "mini" version that is a sample from the larger dataset. So, to determine which archive to try first, we sort them by name, using their edit distance from the word "data." These nested archives then get extracted and processed recursively by going back to Step 3. If any tabular data is found, we exit and return it.

Step 5 If Step 2 is unsuccessful, we look through all ".txt" files and try to determine if they are likely to be tabular data disguised as plain text. We do this by trying various common delimiters (comma, semicolon, and tab) and determining how much of the resulting table is NaN. If the data is "perfect"—i.e. it contains multiple columns, no NaN data, and a regular shape—then we return the resulting dataframe. Otherwise, we keep trying other ".txt" files to find the ones with the lowest proportion of NaN entries and the most regular shape. In the event of a tie, we save all of them.

Step 6 We repeat the same process as in Step 5 for files without an extension, but due to the increased likelihood of false positives, we only accept a data file if it has a lower proportion of NaN entries compared to all the other files we have tried so far. Furthermore, we only save the single best file, rather than multiple files like we did in Step 5.

Step 7 If no likely candidates are identified, we go back to any nested archive. Then, we try to coerce it into tables, by treating nested folders as columns and nested files as entries. If this is unsuccessful, we read the archive as a JSON dictionary mirroring the directory structure (with keys representing file and values representing content), and return this instead.

The flow chart in Fig.Figure 2 provides a more comprehensive view of the logical structure.

Availability

lucie is available via PyPI (`pip install lucie`) and on GitHub at https://github.com/ArnaoutLab/lucie. Forks and contributions are welcome.

Results

The *lucie* algorithm was designed using manually identified patterns found in the top 100 most-popular datasets and tested on datasets ranked 101-250. Dataset imports were considered successful if the following criteria held:

- The algorithm returned a meaningful result, and did not crash, error out, or return "None";

DRAFT

- The first few rows of the data files seemed to contain meaningful data upon visual inspection; and
- The resulting dataframes had many columns and rows (as opposed to just one or a few rows or being empty).

If all three criteria were not met, the import was considered a failure.

As a baseline, *ucimlrepo* succeeded on 84 of the top 100 and failed on the remaining 16 (Fig.Figure 3). In contrast, *lucie* succeeded on all 100 (which effectively served as its training set). As a test, we asked how many of the datasets ranked 101-250 in popularity could be imported by *ucimlrepo* vs. *lucie*. We found that 20 of the datasets in this test set failed for reasons unrelated to importing: 9 did not have valid download links (UCMLR IDs #683, 752, 770, 892, and 465), caused download errors internal to UCMLR (#516), or had a size of 0 bytes (#611, 613, and 346) and 11 were over 100MB which we excluded due to size-related timeouts (UCMLR IDs #442, 920, 344, 791, 779, 773, 256, 231, 226, 203, and 164). We therefore excluded these 9+11=20 datasets from the test set and asked how many of the remaining 150-20=130 datasets could be imported by each utility. *ucimlrepo* successfully imported 95 of these 130 (73.1%), while *lucie* successfully imported 124 out of these 130 (95.4%) (Fig.Figure 3). *lucie* failed on UCMLR IDs #7, 25, 130, 180, 432, and 470.

We also wrote *pytest* tests scripts, which are included in both the GitHub repository and in the PyPI package. We tested all of the common dataset types, and for rare dataset types that we did not encounter in practice, we created synthetic test datasets to demonstrate that *lucie* would support these functions if they were to come up. Our tests result in a code-coverage rate of 98%; the coverage report is included in the GitHub repository. Please see the repository for more information on the tests.

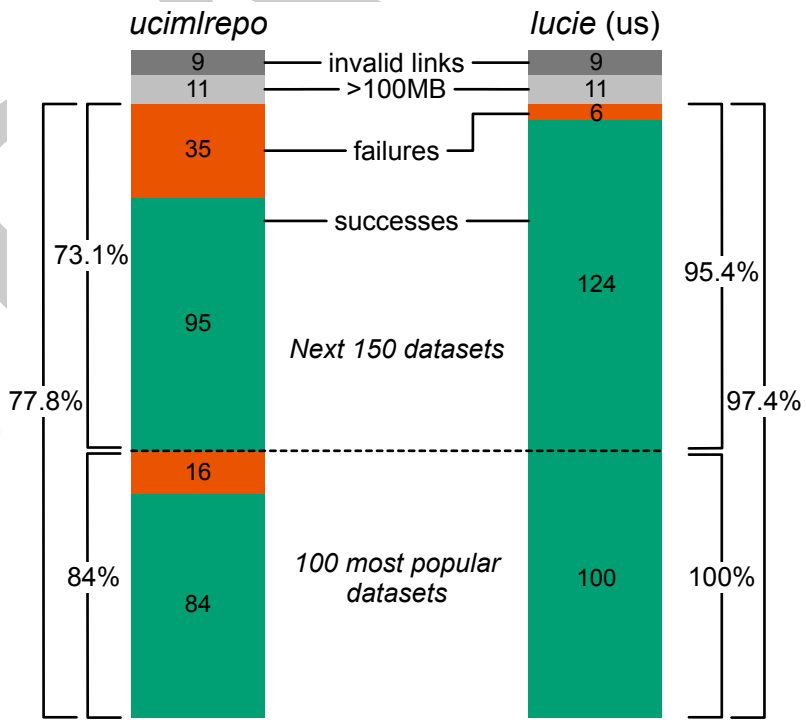

Figure 3: Percent success for ucimlrepo (left) and lucie (right). Green=successful imports; red=failed imports. Percents are shown for the 100 most popular datasets, 130 of the 150 next most popular datasets (separated by dotted line), and these 100+130=230 datasets; the remaining 20 were excluded for having broken/external links (dark gray) and large datasets/timeouts (light gray). Integers in the bars are no. datasets. The 100 most popular datasets, on which lucie was developed, are on the bottom.

Discussion

The breathtaking progress of AI/ML and related fields depends on the ready availability of widely used/high-quality datasets for benchmarking and investigation (Ferreira et al., 2022; Offord, 2023; Yoong et al., 2022). The UCIMLR plays a major role. Frictionless availability of data is critical to open and collaborative science more generally, especially valuable for difficult problems and in times of urgency or crisis (R. Arnaout, 2021; Ramy Arnaout & Arnaout, 2022; Bertram et al., 2023; Bicer et al., 2011; Lakhani et al., 2013; Noy & Goble, 2023). However, these datasets are only useful if they can be easily imported. The diversity and distributed nature of datasets and their contributors understandably and inevitably lead to failure in the default import process, which new code development, such as *lucie*, can ameliorate. We have demonstrated that *lucie* successfully imports the 100 most-popular datasets (as of late 2024), rescuing some failures from the default import method (*ucimlrepo*), and has a high success rate with non-importable datasets outside of this range, as evidenced by 95.4% success rate on datasets 101-250 (excepting broken/outside links and datasets over 100MB, which time out). Future work can help increase the success rate by incorporating more data formats and manually adding cleaned datasets wherever necessary. For example, one potential low hanging fruit could be a more sophisticated way of dealing with ".Z" archives, which caused issues in some of the six datasets in our test set that failed to import.

Future work could analyze the test set more closely, since the analysis of the success rate presented here was based only on the three criteria mentioned and not on, for example, a comprehensive, row-by-row check. An improvement would be to study each dataset more in-depth, and determine manually if any files were missed, or conversely, if *lucie* imported any false positives. Finally, any such codebase requires maintenance, especially as additional datasets are added to the UCIMLR and increase in popularity, and benefits from extension, to make ever more datasets easily useable by the research community. To this end, *lucie* is open source and distributed under the MIT license; any future contributions are welcome. Issues can be left at the GitHub repository.

Conflict of Interest

The authors declare that they have no conflict of interest.

Acknowledgements

This work was supported by the Gordon and Betty Moore Foundation and by the NIH under grants R01HL150394, R01HL150394-SI, R01AI148747, and R01AI148747-SI.

Author Info

Kenneth Ge is a research intern at the Beth Israel Deaconess Medical Center in Boston, MA, USA. He is also an undergraduate studying Computer Science at Carnegie Mellon University in Pittsburgh, PA, USA. He has previous research experience and publications in Human Computer Interaction and Accessible Computing and Machine Learning.

Phuc Nguyen received a B.S. degree in Physics from the University of Texas at Austin in 2010, and a Ph.D. degree in Physics from the University of Texas at Austin in 2017. From 2017 to 2023, he worked as a postdoctoral researcher in physics at the University of Maryland in College Park, the City University of New York, the University of Haifa in Israel and Brandeis University, with research interest spanning from high-energy physics (especially the AdS/CFT correspondence) to general relativity to quantum information science. He is a postdoctoral researcher at Beth Israel Deaconess Medical Center in Boston, MA, USA, with research interest focusing on information theory and machine learning.

DRAFT

Ramy Arnaout received an S.B. degree in biology from the Massachusetts Institute of Technology in 1997, a D.Phil. (Ph.D.) degree in Mathematical Biology (Zoology) from the University of Oxford in 1999, and an M.D. degree with honors from Harvard Medical School in 2003. He is Associate Professor of Pathology at Harvard Medical School and Associate Director of the Clinical Microbiology Laboratories at the Beth Israel Deaconess Medical Center, both in Boston, MA, USA. His past and present research interests include entropy, diversity, immunomics, metagenomics, computational pathology, informatics, artificial intelligence, and machine learning. Dr. Arnaout is a member of the College of American Pathologists.

References

Arnaout, R. (2021). Cooperation under Pressure: Lessons from the COVID-19 Swab Crisis. *Journal of Clinical Microbiology*, *59*(10). https://doi.org/10.1128/JCM.01239-21

Arnaout, Ramy, & Arnaout, R. (2022). Visualizing omicron: COVID-19 deaths vs. Cases over time. *PLOS ONE*, *17*(4), e0265233. https://doi.org/10.1371/journal.pone.0265233

Badr, W. (2019). Top Sources For Machine Learning Datasets. In *Medium*. https://towardsdatascience.com/top-sources-for-machine-learning-datasets-bb6d0dc3378b

Bertram, M. G., Sundin, J., Roche, D. G., Sánchez-Tójar, A., Thoré, E. S. J., & Brodin, T. (2023). Open science. *Current Biology: CB*, *33*(15), R792–R797. https://doi.org/10.1016/j.cub.2023.05.036

Bicer, V., Tran, T., Abecker, A., & Nedkov, R. (2011). KOIOS: Utilizing Semantic Search for Easy-Access and Visualization of Structured Environmental Data. In L. Aroyo, C. Welty, H. Alani, J. Taylor, A. Bernstein, L. Kagal, N. Noy, & E. Blomqvist (Eds.), *The Semantic Web – ISWC 2011* (pp. 1–16). Springer Berlin Heidelberg. ISBN: 978-3-642-25093-4

Černoch, R. (2023). *Cernoch/movies*. https://github.com/cernoch/movies

Chinn, E., Arora, R., Arnaout, R., & Arnaout, R. (2023). ENRICHing medical imaging training sets enables more efficient machine learning. *Journal of the American Medical Informatics Association: JAMIA*, *30*(6), 1079–1090. https://doi.org/10.1093/jamia/ocad055

Dey, D., Arnaout, R., Antani, S., Badano, A., Jacques, L., Li, H., Leiner, T., Margerrison, E., Samala, R., Sengupta, P. P., Shah, S. J., Slomka, P., Williams, M. C., Bandettini, W. P., & Sachdev, V. (2023). Proceedings of the NHLBI workshop on artificial intelligence in cardiovascular imaging: Translation to patient care. *JACC. Cardiovascular Imaging*, *16*(9), 1209–1223. https://doi.org/10.1016/j.jcmg.2023.05.012

Ferreira, D. L., Salaymang, Z., & Arnaout, R. (2022, October 10). *Label-free segmentation from cardiac ultrasound using self-supervised learning*. *arXiv.org*. https://arxiv.org/abs/2210.04979v2

Koch, B., Denton, E., Hanna, A., & Foster, J. (2021). Reduced, reused and recycled: The life of a dataset in machine learning research. *None*.

Lakhani, K. R., Boudreau, K. J., Loh, P.-R., Backstrom, L., Baldwin, C., Lonstein, E., Lydon, M., MacCormack, A., Arnaout, R. A., & Guinan, E. C. (2013). Prize-based contests can provide solutions to computational biology problems. *Nature Biotechnology*, *31*(2), 108–111. https://doi.org/10.1038/nbt.2495

Markelle Kelly, K. N., Rachel Longjohn. (2024). *The UCI machine learning repository*. University of California, Irvine, School of Information; Computer Sciences. https://archive.ics.uci.edu

Mayfield, H., Smith, C., Gallagher, M., & Hockings, M. (2017). Use of freely available datasets and machine learning methods in predicting deforestation. *Environmental Modelling & Software*, *87*, 17–28. https://doi.org/https://doi.org/10.1016/j.envsoft.2016.10.006

DRAFT

Miao, B. Y., Chen, I. Y., Williams, C. Y., Davidson, J., Garcia-Agundez, A., Sun, S., Zack, T., Saria, S., Arnaout, R., Quer, G., Sadaei, H. J., Torkamani, A., Beaulieu-Jones, B., Yu, B., Gianfrancesco, M., Butte, A. J., Norgeot, B., & Sushil, M. (2024, March 5). *The minimum information about CLinical artificial intelligence checklist for generative modeling research (MI-CLAIM-GEN). arXiv.org*. https://arxiv.org/abs/2403.02558v2

Noy, N., & Goble, C. (2023). Are we cobblers without shoes? Making computer science data FAIR. *Communications of ACM*, *66 (1)*. https://cacm.acm.org/magazines/2023/1/267975-are-we-cobblers-without-shoes/abstract

Offord, C. (2023). UK Biobank releases half a million whole-genome sequences for biomedical research. In *ScienceInsider*. https://www.science.org/content/article/uk-biobank-releases-half-million-whole-genome-sequences-biomedical-research

Reuters-text-categorization/reuters_loader.py at master · jchavando/reuters-text-categorization · GitHub. (n.d.). Retrieved October 3, 2024, from https://github.com/jchavando/reuters-text-categorization/blob/master/reuters_loader.py

Richardson, L. (2007). Beautiful soup documentation. *April*.

UCI Diabetes Data Set. (n.d.). Retrieved October 3, 2024, from https://www.kaggle.com/datasets/ealtintas/uci-machine-learning-repository-diabetes-data-set

Uci-ml-repo/ucimlrepo. (2024). uci-ml-repo. https://github.com/uci-ml-repo/ucimlrepo

Ucirvine/reuters21578 · Datasets at Hugging Face. (2023). https://huggingface.co/datasets/ucirvine/reuters21578

Yin, H., & Berger, C. (2017). When to use what data set for your self-driving car algorithm: An overview of publicly available driving datasets. *2017 IEEE 20th International Conference on Intelligent Transportation Systems (ITSC)*, 1–8. https://doi.org/10.1109/ITSC.2017.8317828

Yoong, S. L., Turon, H., Grady, A., Hodder, R., & Wolfenden, L. (2022). The benefits of data sharing and ensuring open sources of systematic review data. *Journal of Public Health (Oxford, England)*, *44*(4), e582–e587. https://doi.org/10.1093/pubmed/fdac031

DRAFT